

Yük Altında Buton Yanıtı

Proje çalışma raporu: ne yaptık, nasıl çalışıyor, neden böyle tasarlandı

Kart: STM32F407G-DISC1 (Discovery), 168 MHz

RTOS: FreeRTOS, CMSIS-RTOS v2 arayüzü (STM32CubeIDE)

PC arayüzü: Python 3 · Tkinter · pyserial · matplotlib

Bağlantı: USART2 (PA2/PA3) → CP2102 USB-TTL → COM3, 115200 8N1

Hazırlayan: Yusuf Kaan İnan (yapay zekâ desteğiyle)

Tarih: 5 Ekim 2026

İçindekiler

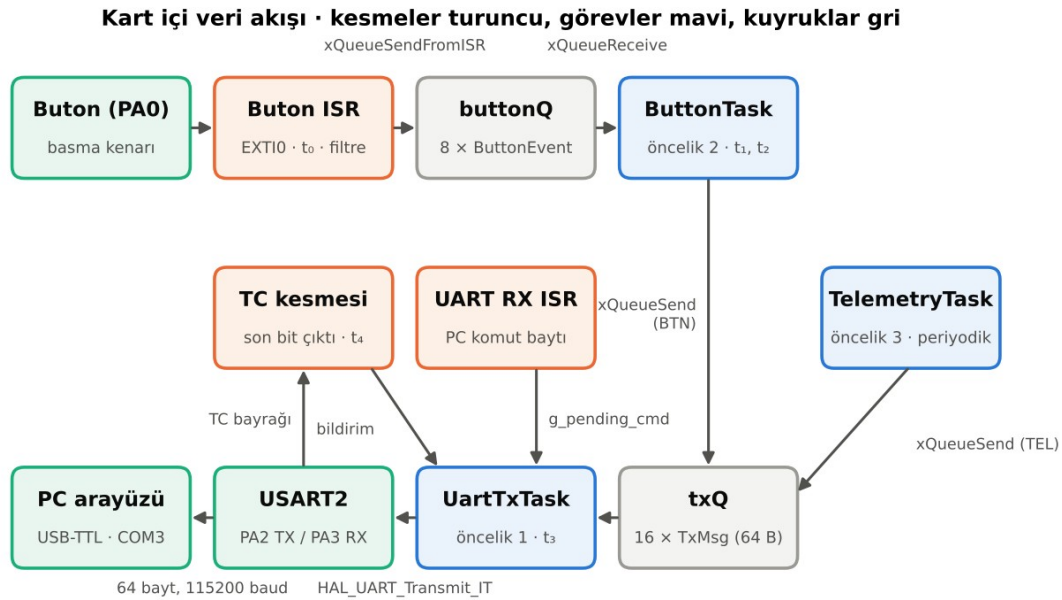
1. Projenin özeti ve mimari
2. RTOS temelleri: görev, scheduler, öncelik
3. Görevler arası iletişim: queue, semaphore, mutex, task notification
4. Kesmeler, NVIC ve FreeRTOS kuralları
5. Donanım ve CubeMX ayarları
6. Firmware: app.c parça parça
7. Kart ile PC arasındaki 64 baytlık protokol
8. Baud değişimi: bağlantı nasıl kopmadan devam ediyor
9. PC arayüzü nasıl çalışıyor
10. Ölçüm sonuçları ve yorum
11. Karşılaştığımız hatalar ve dersleri
12. İyileştirme önerileri
13. Öğrenmen gerekenler ve kontrol soruları
14. Açık kalan işler ve GitHub

1. Projenin özeti ve mimari

Ödevin sorusu tek cümle: **Butona bastım, yanıt ne zaman geldi ve gecikme hangi aşamada oluştu?** Bunun için kartta üç görevli bir FreeRTOS uygulaması, PC'de de bir arayüz yazdık. Butona her basışta kart, olayın yolculuğunu beş noktada mikrosaniye hassasiyetle ölçüyor ($t_0 \dots t_4$). Telemetry hızını ve CPU yükünü değiştirerek altı senaryoda (S0–S5) bu sürelerin nasıl değiştiğini inceledik.

Kısaca sistem şöyle çalışıyor:

1. Butona basılınca **kesme (ISR)** zamanı not alır (t_0) ve olayı **buton kuyruğuna** atar.
2. **ButtonTask** olayı alır (t_1), "butona basıldı" mesajını hazırlar ve **TX kuyruğuna** bırakır (t_2).
3. Bu sırada **TelemetryTask** da periyodik olarak kendi mesajlarını aynı TX kuyruğuna bırakıyor.
4. **UartTxTask**, UART'ın tek sahibi olarak kuyruktan sırayla mesaj alır, kesmeli gönderimi başlatır (t_3) ve son bit hattan çıkınca gelen **TC kesmesiyle** uyanır (t_4).
5. Deney bitince PC "D" komutu gönderir; kart tüm ölçüm kayıtlarını PC'ye aktarır, arayüz bunları CSV'ye yazar ve grafik çizer.



Şekil 1 · Kart içindeki veri akışı

Bileşen	Ne iş yapar?	Öncelik
TelemetryTask	Periyodik telemetry mesajı üretir; S4/S5'te ek CPU işi yapar	Yüksek (3) · osPriorityAboveNormal
ButtonTask	Buton olayını alır, BTN yanıtını üretir	Orta (2) · osPriorityNormal
UartTxTask	UART'ın tek sahibi; kuyruktaki mesajları gönderir, PC komutlarını işler	Düşük (1) · osPriorityBelowNormal
Buton ISR	t_0 'ı alır, titreşimi filtreler, olayı kuyruğa atar	Kesme (görev değil)
TC kesmesi	Son bit çıkınca t_4 'ü yazar, UartTxTask'ı uyandırır	Kesme
UART RX kesmesi	PC'den gelen tek baytlık komutu alır	Kesme

Önemli: Şartname öncelikleri "3 > 2 > 1" diye verir. CMSIS-RTOS v2'de sayılar farklıdır (AboveNormal = 32, Normal = 24, BelowNormal = 16) ama **sıralama ilişkisi aynıdır**; önemli olan bu ilişkidir.

2. RTOS temelleri: görev, scheduler, öncelik

2.1 RTOS nedir, neden kullanıyoruz?

RTOS'suz (bare-metal) bir programda her şey tek bir `while(1)` döngüsündedir: butona bak, telemetri gönder, UART'ı kontrol et... Bir iş uzun sürerse diğerleri bekler. **RTOS**, programı birbirinden bağımsız **görevlere (task)** böler ve hangi görevin ne zaman çalışacağına **scheduler** (zamanlayıcı) karar verir. Her görev sanki işlemcinin tek sahibiymiş gibi kendi sonsuz döngüsünü yazar.

Gerçek zamanlı demek "hızlı" demek değildir; **öngörülebilir** demektir. Önemli olan, kritik bir işin en kötü durumda ne kadar gecikeceğini bilebilmek. Bu ödev tam olarak bunu ölçüyor.

2.2 Görev (task) ve durumları

Her görevin kendi **stack'i** (yerel değişkenleri ve çağrı zinciri için bellek) ve bir **öncelik** değeri vardır. Bir görev her an şu dört durumdan birindedir:

Durum	Anlamı	Bizim projede örnek
Running	Şu an CPU'da çalışıyor (tek çekirdekte aynı anda yalnız bir görev)	TelemetryTask S5'te 5 ms hesap yaparken
Ready	Çalışmaya hazır, ama daha öncelikli biri CPU'da	TC gelince uyanan ama hesabın bitmesini bekleyen UartTxTask
Blocked	Bir şeyi bekliyor: kuyruk, süre, bildirim. CPU harcamaz	<code>xQueueReceive(buttonQ, ..., portMAX_DELAY)</code> içindeki ButtonTask
Suspended	Açıkça durdurulmuş	Kullanmıyoruz

Hiçbir görev hazır değilse FreeRTOS'un kendi **Idle görevi** çalışır. Debugger'da kartı durdurduğunda `prvIdleTask / prvCheckTasksWaitingTermination` içinde görmek bu yüzdendi: kart çökmemişti, sadece yapacak işi yoktu.

2.3 Scheduler nasıl karar veriyor? (Preemptive, öncelikli)

FreeRTOS'u **preemptive** (kesintili) modda kullanıyoruz. Kural basit: **Ready durumdaki en yüksek öncelikli görev çalışır**. Daha öncelikli bir görev hazır hale geldiği anda, çalışan görev **anında** durdurulur ve CPU ona verilir. Buna **preemption** denir.

Scheduler iki durumda devreye girer:

- **Tick kesmesi (SysTick, 1 ms'de bir):** Zaman sayacı artar; süresi dolan `vTaskDelay/vTaskDelayUntil` beklemleri biter, uyanan görevler Ready olur.
- **Bir olay bir görevi uyandırınca:** Kuyruğa veri gelmesi, bildirim, vb. Kesme içinden olduysa `portYIELD_FROM_ISR` ile, görevden olduysa API çağrısının içinde.

Görev değiştirme işinin kendisine **context switch** denir. Cortex-M'de bu iş **PendSV** kesmesinde yapılır: çalışan görevin yazmaçları kendi stack'ine kaydedilir, yeni görevinkiler yüklenir. Bu birkaç mikrosaniye sürer; ölçümlerdeki $t_1 - t_0 \approx 12 \mu s$ 'nin bir kısmı budur.

Bir buton basışında scheduler adım adım ne yapıyor? (S3 örneği)

1. UartTxTask bir TEL gönderimi başlatmış, TC'yi bekliyor (Blocked). TelemetryTask bir sonraki periyodu bekliyor (Blocked). CPU'da Idle var.
2. Butona basılıyor → EXTI0 kesmesi. ISR, t_0 'ı alır, olayı `xQueueSendFromISR` ile buton kuyruğuna atar. Bu çağrı "ButtonTask uyandı ve Idle'dan öncelikli" bilgisini **woken** değişkeniyle döndürür.

3. `portYIELD_FROM_ISR(woken)` PendSV'yi tetikler; ISR biter bitmez ButtonTask çalışır (t_1 , t_0 'dan ~12 μ s sonra).
4. ButtonTask mesajı hazırlar, TX kuyruğuna koyar ve tekrar Blocked olur. Ama UART hâlâ TEL'i gönderiyor; BTN kuyrukta bekler. t_3-t_2 **büyümesinin sebebi budur**.
5. TEL'in son biti çıkınca TC kesmesi UartTxTask'ı uyandırır; görev kuyruktan BTN'i alır, gönderimi başlatır (t_3), 5,56 ms sonra TC tekrar gelir (t_4).

2.4 Periyodik görev: `vTaskDelay` ile `vTaskDelayUntil` farkı

`vTaskDelay(10)` "şimdiden itibaren 10 ms uyu" der; görevin kendi çalışma süresi periyoda eklenir ve periyot kayar. `vTaskDelayUntil(&last, 10)` ise "bir önceki uyanıştan **tam** 10 ms sonra uyan" der; periyot sabit kalır. TelemetryTask'ta bu yüzden ikincisini kullandık. İlk testte TEL zaman damgalarının tam 100 ms arayla artması (19300, 19400, 19500...) bunu kanıtladı.

app.c · App_TelemetryTask (özet)

```
TickType_t last = xTaskGetTickCount();
for (;;) {
    /* ... mesajı üret, kuyruğa bırak ... */
    vTaskDelayUntil(&last, pdMS_TO_TICKS(s->period_ms));
}
```

Senaryo değiştiğinde `last`'ı sıfırlıyoruz; yoksa `vTaskDelayUntil`, görev uyurken "kaçırdığı" periyotları art arda telafi etmeye çalışırdı.

3. Görevler arası iletişim: queue, semaphore, mutex, task notification

Görevler aynı belleği paylaşır. İki görev aynı değişkene aynı anda yazarsa veri bozulur (**race condition**). FreeRTOS bu yüzden güvenli iletişim araçları sunar. Dördünü de bilmek gerekiyor; projede ikisini (queue ve task notification) kullandık, mutex'i bilinçli olarak kullanmadık.

3.1 Queue (kuyruk)

Queue, sabit boyutlu elemanlardan oluşan bir **FIFO** (ilk giren ilk çıkar) tampondur. Bir taraf `xQueueSend` ile koyar, diğer taraf `xQueueReceive` ile alır. Üç önemli özelliği var:

- **Değer olarak kopyalar.** Kuyruğa verinin adresi değil, kendisi kopyalanır. Gönderen taraf yerel değişkenini hemen yeniden kullanabilir. Şartnamedeki "yerel tampon adresinin ömrüne güvenmeyin" uyarısı bu yüzden; biz 64 baytlık mesajın tamamını kopyalıyoruz.
- **Bloklama.** Alıcı kuyruk boşsa `portMAX_DELAY` ile **Blocked** olup bekler, CPU harcamaz. Gönderen kuyruk doluysa beklemeyi ya da hemen vazgeçmeyi (timeout = 0) seçer. Biz 0 kullanıyoruz: kuyruk doluysa mesaj düşer ve **sayılır**.
- **Kesmeden kullanım.** ISR içinde normal API kullanılamaz; `...FromISR` sürümleri vardır (`xQueueSendFromISR`). Bunlar asla bloklamaz ve uyanan görev daha öncelikliyse bunu bildirir.

Kuyruk nasıl oluşuyor? `xQueueCreate(eleman_sayısı, eleman_boyutu)` FreeRTOS heap'inden `eleman_sayısı × eleman_boyutu` + kontrol bloğu kadar yer ayırır. Bizim iki kuyruğumuz:

```

void App_Init(void)
{
    txQ = xQueueCreate(TXQ_LEN, sizeof(TxMsg));          /* 16 × 68 bayt */
    configASSERT(txQ != NULL);

    buttonQ = xQueueCreate(BTNQ_LEN, sizeof(ButtonEvent)); /* 8 × 8 bayt */
    configASSERT(buttonQ != NULL);

    HAL_TIM_Base_Start(&htim2); /* µs saatini başlat */
}

```

Kuyruk	Eleman	Kim koyar?	Kim alır?
buttonQ (8)	ButtonEvent {id, t0}	Buton ISR (xQueueSendFromISR)	ButtonTask
txQ (16)	TxMsg {64 bayt metin, tip, olay id}	TelemetryTask, ButtonTask	UartTxTask

3.2 Semaphore (semafor)

Semafor bir **sinyal** aracıdır; veri taşımaz, sadece "bir şey oldu" ya da "şu kadar kaynak var" der.

- **Binary semaphore:** 0 ya da 1. Tipik kullanım: kesme **Give** eder, görev **Take** ile bekler. "Olay oldu, uyan" demenin klasik yolu.
- **Counting semaphore:** 0...N. Örneğin 3 adet tampondan kaçının boş olduğunu saymak.

Semaforun **sahibi yoktur**: bir görev alıp başka bir görev ya da kesme verebilir.

3.3 Mutex

Mutex (mutual exclusion) **paylaşılan bir kaynağı korumak** içindir: "bu kaynağı şu an ben kullanıyorum, bitene kadar kimse dokunmasın." Görünüşte binary semafora benzer ama iki kritik farkı vardır:

- **Sahiplik:** Mutex'i alan görev onu vermek zorundadır. Kesmeden kullanılamaz.
- **Öncelik kalıtımı (priority inheritance):** Düşük öncelikli görev mutex'i tutarken yüksek öncelikli görev onu beklerse, düşük görevin önceliği geçici olarak yükseltilir. Bu, **priority inversion** denen tehlikeyi azaltır: orta öncelikli bir görevin araya girip yüksek öncelikliyi dolaylı yoldan bekletmesi.

	Binary semaphore	Mutex	Queue	Task notification
Amaç	Olay sinyali	Kaynağı korumak	Veri taşımak	Hafif sinyal / sayaç
Veri taşır mı?	Hayır	Hayır	Evet (kopyalayarak)	32 bit değer
Sahibi var mı?	Hayır	Evet	Hayır	Hedef görev sabit
Kesmeden kullanılır mı?	Evet (Give)	Hayır	Evet (FromISR)	Evet (FromISR)
Öncelik kalıtımı	Yok	Var	Yok	Yok
Bizde kullanıldı mı?	Hayır	Hayır (bilinçli)	Evet: buttonQ, txQ	Evet: TC → UartTxTask

Neden mutex kullanmadık?

UART'ı iki görev doğrudan kullansaydı (TelemetryTask ve ButtonTask), UART'ı bir mutex ile korumamız gerekirdi. Bunun yerine şartnamenin önerdiği daha iyi bir tasarım seçtik: **UART'ın tek sahibi UartTxTask**. Diğer görevler UART'a hiç dokunmuyor, sadece kuyruğa mesaj bırakıyor. Paylaşılan

kaynak olmayınca mutex'e de, priority inversion riskine de gerek kalmıyor. Kuyruk hem iletişimi hem de sıralamayı (serileştirmeyi) çözüyor.

3.4 Task notification (görev bildirimi)

Her FreeRTOS görevinin içinde hazır bir bildirim sayacı vardır. Bir kesme ya da görev ona `vTaskNotifyGiveFromISR` ile "haber" verir, görev de `ulTaskNotifyTake` ile bekler. **Binary semaforun daha hızlı ve daha az RAM kullanan hali** gibi düşünebilirsin; tek kısıtı, alıcının sabit tek bir görev olmasıdır. Ayrı bir nesne oluşturmaya gerek yoktur.

Biz bunu "UART gönderimi bitti" sinyali için kullanıyoruz:

app.c · uart_send_current ve HAL_UART_TxCpltCallback

```
/* UartTxTask: gönderimi başlat, bitene kadar (en fazla 1 s) uyu */
if (HAL_UART_Transmit_IT(&huart2, (uint8_t *)tx_cur.data, MSG_LEN) != HAL_OK) {
    cnt_tx_err++; ... return;
}
if (ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(TX_TIMEOUT_MS)) == 0U) {
    HAL_UART_AbortTransmit(&huart2); /* 1 s'de bitmediyse güvenli durdur */
    cnt_tx_timeout++;
}

/* TC kesmesi: son bit çıktı */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    const uint32_t now = timer_us(); /* t4 */
    ...
    BaseType_t woken = pdFALSE;
    vTaskNotifyGiveFromISR(txTaskHandle, &woken);
    portYIELD_FROM_ISR(woken);
}
```

3.5 Kritik bölge (critical section)

Yeni senaryo başlarken onlarca sayacı sıfırlıyoruz. Bu sırada buton kesmesi gelip aynı sayaçları artırırsa sıfırlama yarım kalır. `taskENTER_CRITICAL()` ... `taskEXIT_CRITICAL()` arası kısa süre için kesmeleri (FreeRTOS'un yönettiği öncelik seviyesine kadar) kapatır. **Kritik bölge çok kısa tutulmalıdır**; içinde bekleme, UART, printf olmaz.

app.c · start_scenario

```
taskENTER_CRITICAL();
g_scenario = scn;
btn_next_id = 0U;
memset(recs, 0, sizeof recs);
cnt_btn_accepted = cnt_btn_bounce = ... = 0U;
g_work_iters = iters_for_us(k_scenarios[scn].work_us);
taskEXIT_CRITICAL();
```

4. Kesmeler, NVIC ve FreeRTOS kuralları

Kesme (interrupt), donanımın "şimdi bana bak" demesidir. CPU o an ne yapıyorsa bırakır, **ISR** (kesme servis rutini) çalışır, sonra kaldığı yere döner. ISR bir görev **değildir**; scheduler'ın dışında, tüm görevlerden önce çalışır.

4.1 ISR'ı kısa tutmak

ISR çalışırken diğer görevler ve düşük öncelikli kesmeler bekler. Bu yüzden buton ISR'ımız sadece zamanı alıyor, filtreliyor ve olayı kuyruğa atıyor. Mesajı hazırlamak, UART'a yazmak gibi işler ButtonTask'ta. ISR içinde **asla** bekleme, **printf**, **HAL_Delay** ya da bloklayan FreeRTOS çağrısı olmaz.

4.2 NVIC öncelikleri ve FreeRTOS

ARM Cortex-M'de **küçük sayı = yüksek öncelik** (0 en acil, 15 en sakın). FreeRTOS iki ayar ister:

Ayar	Değer	Anlamı
LIBRARY_LOWEST_INTERRUPT_PRIORITY	15	SysTick ve PendSV en düşük öncelikte çalışır; gerçek olaylar onları her zaman bölebilir
LIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY	5	FreeRTOS fonksiyonu çağıran kesmeler 5–15 arasında olmalı. 0–4 arası FreeRTOS'a dokunamaz
EXTI0 (buton), USART2	5	İkisi de ...FromISR çağırdığı için 5 veya daha büyük
Priority Group	4 bit preemption, 0 bit sub	FreeRTOS tüm öncelik bitlerinin preemption için kullanılmasını ister

Bu kurallara uyulmazsa sistem **rastgele kilitlenir** ve bu, bulması en zor hatalardandır. CubeMX'in "SysTick önceliği 15 olsun mu?" uyarısına bu yüzden Evet dedik.

4.3 FromISR ve portYIELD_FROM_ISR

Kesmeden kuyruğa veri koyunca bekleyen bir görev uyanabilir. Bu görev, kesilen görevden daha öncelikliyse kesme biter bitmez ona geçilmelidir; yoksa bir sonraki tick'e (1 ms) kadar beklerdi. **xQueueSendFromISR** bunu **woken** değişkeniyle bildirir, **portYIELD_FROM_ISR(woken)** da geçişi hemen yaptırır. Ölçümde $t_1 - t_0$ 'nın $\sim 12 \mu s$ olması (1 ms değil) bunun kanıtı.

5. Donanım ve CubeMX ayarları

Bölüm	Ayar	Neden?
RCC / Saat	HSE kristal, HCLK 168 MHz	Kartın en yüksek hızı; doğru baud için kararlı saat şart
SYS	Debug: Serial Wire · Timebase: TIM6	SysTick FreeRTOS'a ait; HAL ayrı bir zaman kaynağı kullanır
USART2	Asynchronous, 115200, 8N1, global interrupt ✓	PA2 = TX, PA3 = RX; kesmeli gönderim/alım
PA0 (mavi buton)	GPIO_EXTI0, Rising/Falling, No pull	Kartta harici pull-down ve RC filtre var; iki kenar filtre için gerekli
TIM2	Internal clock, Prescaler 83, Period 4294967295	$84 \text{ MHz} / 84 = 1 \text{ MHz} \rightarrow 1 \text{ sayı} = 1 \mu s$; 32 bit, $\sim 71,6 \text{ dk'da}$ bir başa döner
NVIC → Code generation	Enable in Init, IRQ handler, Call HAL handler ✓	İşaretlenmezse kesme hiç açılmaz (yaşadık)
FreeRTOS	CMSIS_V2, 3 task (dinamik), defaultTask silindi	Şartname tam 3 uygulama görevi istiyor

Bölüm	Ayar	Neden?
FREERTOS	UartTxTask stack 512 word, CHECK_FOR_STACK_OVERFLOW Option2	Kayıt biçimlendirme stack ister; taşma olursa yakalansın
FREERTOS	USE_NEWLIB_REENTRANT Enabled	snprintf farklı görevlerden güvenli çağrılabilir
Derleme	Debug (-O0)	Tüm senaryolar aynı derlemeyle ölçüldü; kalibrasyon buna göre

Kablolama

CP2102	Discovery	Not
TXD	PA3 (USART2 RX)	Çapraz bağlanır: birinin konuştuğu ucu diğeri dinler
RXD	PA2 (USART2 TX)	
GND	GND	Ortak referans şart; yoksa anlamsız karakterler
3V3 / 5V	Bağlanmaz	Kart enerjisini kendi USB'sinden alır

Dosya düzeni: CubeMX'in ürettiği **main.c** sadece bizim fonksiyonlarımızı çağırır. Uygulama mantığının tamamı **app.c** / **app.h**'dedir. Böylece kod yeniden üretilse bile bizim kodumuz bozulmaz.

main.c · bağlantı noktaları (özet)

```
/* main.c */
/* USER CODE BEGIN RTOS_QUEUES */
App_Init();
/* USER CODE END RTOS_QUEUES */

void StartTelemetryTask(void *argument) { App_TelemetryTask(); }
void StartButtonTask(void *argument) { App_ButtonTask(); }
void StartUartTxTask(void *argument) { App_UartTxTask(); }
```

6. Firmware: app.c parça parça

6.1 Telemetri nedir? TelemetryTask

Telemetri, bir cihazın kendi durumunu uzaktaki bir izleyiciye **düzenli aralıklarla** göndermesidir (uçaklardaki uçuş verisi, bir motorun sıcaklığı gibi). Bizim ödevde içeriği önemli değil; amaç **yük oluşturmak**: UART hattını ve CPU'yu meşgul ederek buton yanıtının nasıl etkilendiğini görmek. Her TEL mesajı bir sıra numarası, senaryo adı ve kartın açılıştan beri geçen süresini taşır:

TEL,1042,S3,726.

app.c · App_TelemetryTask (sadeleştirilmiş)

```
for (;;) {
    const Scenario *s = &k_scenarios[g_scenario];

    /* Telemetri kapalıysa (S0 ya da deney bitti) BLOKLAN, boş döngüde dönme */
    if (!g_tel_enabled || s->period_ms == 0U) {
        (void)ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        last = xTaskGetTickCount();
        continue;
    }

    if (g_work_iters != 0U) {                /* S4/S5: ek CPU işi */
        g_work_sink ^= cpu_work(g_work_iters);
    }

    TxBMsg m;
    if (make_msg(&m, MSG_TEL, 0U, "TEL,%lu,%s,%lu", ++tel_seq, s->name, now_ms)) {
        if (xQueueSend(txQ, &m, 0) == pdPASS) cnt_tel_sent++;
        else cnt_tel_drop++; /* kuyruk dolu */
    }
    vTaskDelayUntil(&last, pdMS_TO_TICKS(s->period_ms));
}
```

S0'da telemetri kapalıyken görev **bloklanıyor**, boş döngüde dönmüyor. Şartname bunu özellikle istiyor: "kapalı" görev CPU harcamamalı.

6.2 S4/S5 için CPU yükü ve kalibrasyon

Ek yük **gerçek bir hesaplama** olmalı. **vTaskDelay(2)** yük üretmez; sadece uyur ve CPU'yu başkasına bırakır. Bu yüzden sabit tekrarlı bir xorshift hesabı kullanıyoruz. Sonucu **volatile** bir değişkene yazıyoruz ki derleyici "bu sonuç kullanılmıyor" deyip hesabı silemesin.

Kaç tekrarın 2 ms ettiği karta ve derleme ayarına göre değişir. Kart açılıştaki 100 000 tekrarı 5 kez ölçer, **en kısa** süreyi alır (araya kesme girerse süre uzar; en kısa ölçüm en temizdir) ve tekrar sayısını buna göre hesaplar. Gerçek iş süresi her periyotta ayrıca ölçülüp **WRK** satırıyla raporlanıyor: S4'te ortalama 2001 µs, S5'te 5016 µs ölçüldü.

app.c · cpu_work

```
static uint32_t __attribute__((noinline)) cpu_work(uint32_t iters)
{
    uint32_t x = 2463534242U;
    for (uint32_t i = 0U; i < iters; i++) {
        x ^= x << 13; x ^= x >> 17; x ^= x << 5;
    }
    return x;
}
```

6.3 Buton ISR ve titreşim (debounce) filtresi

Mekanik buton basılıp bırakılırken kontaklar titrer ve bir basış onlarca kenar üretir. Discovery kartında butonun yanında bir RC devresi var; bırakırken kondansatör yavaş boşalır ve pin voltajı eşğin etrafında uzun süre gezinip **çok sayıda gürültü kenarı** üretir. Ölçümlerle filtremizi üç adımda geliştirdik:

Sürüm	Kural	Sonuç (10 basış)
1	Son kabul edilen kenardan 30 ms içindekileri at	25 olay, 499 titreşim: bırakma gürültüsü yeni basış sanıldı
2	Rising/Falling + durum takibi (basılı / bırakılmış)	10/10, ama arada bir 116–120 ms'de sahte basış
3	Sessizlik (önceki 30 ms'de hiç kenar yok) + 250 ms basış kilidi	10/10, kararlı. 60 basışlık S3 deneyinde kilit 57 sahte basışı engelledi

app.c · HAL_GPIO_EXTI_Callback (özet)

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    const uint32_t now = timer_us();          /* t0 adayı: İLK iş */
    ...
    const bool level = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET);
    const bool quiet = !has_edge || (uint32_t)(now - last_edge_us) >= DEBOUNCE_US;
    has_edge = true; last_edge_us = now; /* her kenar pencereyi yeniden başlatır */

    if (!(level && quiet)) { cnt_btn_bounce++; return; } /* gürültü */
    if (has_press && (uint32_t)(now - last_press_us) < LOCKOUT_US) {
        cnt_btn_lockout++; return; /* 250 ms kilit */
    }
    ...
    rec_set(r, 0U, now); /* t0 */
    ButtonEvent e = { .id = id, .t0 = now };
    if (xQueueSendFromISR(buttonQ, &e, &woken) != pdPASS) cnt_btn_q_drop++;
    portYIELD_FROM_ISR(woken);
}
```

Gerçek basışın ilk kenarı uzun bir sessizlikten sonra gelir ve **hemen** kabul edilir; yani filtre t_0 'ı geciktirmez. Gürültü kenarlarının önünde ise hep başka kenarlar olduğundan hiçbiri "sessizlikten sonra" gelmez. Basışlar arası kilit, prosedürün en az 0,5 s aralık istemesine dayanıyor; kilide takılanlar **kilit** sayacıyla gizlenmeden raporlanıyor.

Dikkat: Bir olayın t_0 'ı **tek bir global değişkende** tutulmuyor; her olay kendi kimliği ve t_0 'ı ile kuyruğa **kopyalanıyor**. Hızlı iki basışta global değişken kullansaydık ikincisi birincinin üzerine yazardı.

6.4 ButtonTask

app.c · App_ButtonTask (özet)

```
for (;;) {
    xQueueReceive(buttonQ, &e, portMAX_DELAY);
    const uint32_t t1 = timer_us(); /* t1: olayı aldıktan HEMEN sonra */
    rec_set(rec_get(e.id), 1U, t1);

    TxMsg m;
    make_msg(&m, MSG_BTN, e.id, "BTN,%u,%s,PRESSED", e.id, s->name);

    rec_set(rec_get(e.id), 2U, timer_us()); /* t2: xQueueSend'den HEMEN önce */
    if (xQueueSend(txQ, &m, 0) != pdPASS) {
        cnt_btn_tx_drop++; r->status = ST_TX_DROP; /* kayıp gizlenmez */
    }
}
```

6.5 UartTxTask: UART'ın tek sahibi

Bu görev üç iş yapar: kuyruktaki mesajları sırayla gönderir, PC'den gelen komutları çalıştırır ve UART alımının canlı kalmasını sağlar.

app.c · App_UartTxTask

```
for (;;) {
    rx_keeplive(); /* komut alımı kapanmışsa aç */

    if (xQueueReceive(txQ, &tx_cur, pdMS_TO_TICKS(CMD_POLL_MS)) == pdPASS) {
        uart_send_current(); /* t3 ... TC ... t4 */
        continue;
    }
    /* Kuyruk BOŞ: bekleyen PC komutu varsa şimdi çalıştır */
    const char c = g_pending_cmd;
    if (c != 0) { g_pending_cmd = 0; handle_command(c); }
}
```

Kesmeli (IT) gönderim: DR, shift register, TXE ve TC

HAL_UART_Transmit_IT gönderimi **başlatır ve hemen döner**; CPU baytları tek tek beklemez. UART donanımında iki yazmaç vardır:

- **DR (data register):** CPU'nun bir sonraki baytı bıraktığı yer.
- **Shift register:** Baytı DR'den alıp bitlerini tek tek TX hattına kaydıran donanım. Her bayt 10 bit: 1 başlangıç + 8 veri + 1 durma.

TXE kesmesi "DR boşaldı, sıradaki baytı ver" der; HAL her TXE'de bir bayt yazar. **TC** (Transmission Complete) ise "shift register da boşaldı, son durma biti hattan çıktı" der. t_4 'ü TC'de alıyoruz, çünkü şartname "DMA/TXE bitti" ile "son bit çıktı"nın aynı şey olmadığını vurguluyor. Ölçülen $t_4 - t_3 = 5,551 - 5,561$ ms; teorik değer $64 \times 10 / 115200 = 5,556$ ms. Ölçüm sistemimizin doğruluğunu bu uyum kanıtlıyor.

Gönderilen tampon (**tx_cur**) **static**'tir. IT gönderimi arka planda bu tamponu okur; TC gelene kadar yerinde ve değişmeden kalması şarttır. Yerel bir dizi olsaydı fonksiyon dönünce içeriği bozulabilirdi.

6.6 Ölçüm noktaları ve kayıt kartı

Her buton olayı için 64 elemanlı bir havuzda bir **kayıt kartı (Record)** var. Kart zincir boyunca elden ele geçer; her zaman alanını tek bir yer yazar. Bu yüzden kilit gerekmez.

Nokta	Kim yazar?	Nerede?	Aralığın anlamı
t_0	Buton ISR	Kenar kabul edilince, ISR'in ilk satırı	Fiziksel basma anı değil; kesme girişi
t_1	ButtonTask	xQueueReceive 'den hemen sonra	$t_1 - t_0$: kuyruk aktarımı + bağlam değişimi + CPU bekleme
t_2	ButtonTask	xQueueSend 'den hemen önce	$t_2 - t_1$: mesajı hazırlama (vsprintf)
t_3	UartTxTask	HAL_UART_Transmit_IT 'ten hemen önce	$t_3 - t_2$: TX kuyruğunda bekleme (UART sırası + CPU)
t_4	TC kesmesi	HAL_UART_TxCpltCallback ilk satırı	$t_4 - t_3$: 64 baytın hatta geçişi + TC gözlemi

Zamanlar 32 bitlik TIM2 sayacından okunuyor. Farklar hep **uint32_t** çıkarmayla alınıyor; sayaç başa dönse bile (mod 2^{32}) sonuç doğru çıkar. Ölçülemeyen bir zaman (ör. kuyruk dolu olduğu için t_3 , t_4 yok) **boş** bırakılır, asla 0 yazılmaz.

6.7 Komutlar ve bir deneyin akışı

PC tek baytlık komutlar gönderir. RX kesmesi komutu alınca telemetriyi ve basış kaydını **hemen durdurur**, komutu `g_pending_cmd` değişkenine yazar. UartTxTask kuyruktaki mesajları bitirince komutu çalıştırır. Böylece şartnamedeki "önce telemetriyi durdur, TX'i tamamla, sonra kayıtları aktar" kuralı kendiliğinden sağlanıyor.

Komut	Ne yapar?
0 ... 5	Senaryo S0...S5'i seçer, kayıtları ve sayaçları sıfırlar, INF gönderir, telemetriyi başlatır
D	Deneyi bitirir: REC satırları (olay başına bir), WRK, SUM, END
a ... f	Baud değiştirir: 9600, 57600, 115200, 230400, 460800, 921600 (Bölüm 8)

Bir deneyin akışı (senin gerçek S3 kaydından)

```
PC → '3'                S3'ü seç, sıfırla
Kart→ INF,S3,10,0,0,20891,115200
Kart→ TEL,1,S3,12 TEL,2,S3,22 ... (5 s ısınma)
      ... ≥ 30 basış: her birinde BTN,<id>,S3,PRESSED ...
PC → 'D'
Kart→ REC,S3,1,231632142,12,30,57,5609,ok (her olay için)
Kart→ WRK,S3,0,0,0,0,0
Kart→ SUM,S3,60,474,0,0,0,0,0,3084,0,1,0,57,0
Kart→ END,60
PC : measurements/S3.csv + summary.csv + raw/S3_<tarih>.log
```

7. Kart ile PC arasındaki 64 baytlık protokol

Kart C ile, arayüz Python ile yazıldı. İkisinin anlaşabilmesi için önce ortak dili ([docs/protocol.md](#)) yazıya döktük.

7.1 Neden her mesaj tam 64 bayt?

- **Şartname istiyor** ve bunun mühendislik nedeni var: her mesajın hat süresi sabit olur (5,556 ms). Senaryolar arasındaki fark mesaj boyundan değil scheduling'den gelir.
- **Çerçeveleme**: Seri port veriyi parça parça verir. PC, `\n` (LF) görene kadar biriktirir; mesajın nerede bittiğini böyle bilir. Ardından uzunluğun 64 olduğunu kontrol eder. 64 olmayan satır bozuk sayılır.
- **Sessiz kesme yok**: 63 karaktere sığmayan metin kesilmez, hiç gönderilmez ve `fmt_err` sayacı artar.

app.c · make_msg

```
static bool make_msg(TxMsg *m, MsgType type, uint16_t id, const char *fmt, ...)
{
    char tmp[MSG_LEN];
    int n = vsnprintf(tmp, sizeof tmp, fmt, ap);
    if (n < 0 || n > (int)(MSG_LEN - 1U)) { cnt_fmt_err++; return false; }

    memset(m->data, ' ', MSG_LEN - 1U); /* 63 baytı boşlukla doldur */
    memcpy(m->data, tmp, (size_t)n);      /* metni başa koy */
    m->data[MSG_LEN - 1U] = '\n';         /* 64. bayt: LF */
    ...
}
```


Adım	Kart	PC arayüzü
6	300 ms bekler (PC'ye zaman), yeni hızla BAU, 230400, OK gönderir	OK'yi okur → değişim doğrulandı
7	Aynı senaryoyu start_scenario ile yeni hızla baştan başlatır : sayaçlar sıfırlanır, INF gönderilir, telemetri tekrar akar	INF gelir, yeni deney başlar. Etiket: S3_b230400
Hata		2 s içinde SWITCH gelmezse eski hıza döner, kullanıcıyı uyarır

app.c · change_baud

```
static void change_baud(uint32_t baud)
{
    make_msg(&tx_cur, MSG_BAU, 0U, "BAU,%lu,SWITCH", baud);
    uart_send_current(); /* ESKİ hızla, TC'ye kadar bekler */

    HAL_UART_AbortReceive(&huart2);
    huart2.Init.BaudRate = baud;
    if (HAL_UART_Init(&huart2) != HAL_OK) {
        cnt_uart_err++;
        huart2.Init.BaudRate = g_baud; /* başarısızsa eski hıza dön */
        (void)HAL_UART_Init(&huart2);
    } else {
        g_baud = baud;
    }
    HAL_UART_Receive_IT(&huart2, &rx_byte, 1U);

    vTaskDelay(pdMS_TO_TICKS(300)); /* PC portunu değiştirsin */
    make_msg(&tx_cur, MSG_BAU, 0U, "BAU,%lu,OK", g_baud);
    uart_send_current(); /* YENİ hızla */

    start_scenario(g_scenario); /* telemetri yeni hızla yeniden başlar */
}
```

Kart her açılışta (ya da reset sonrası) **115200** ile başlar. Kart resetlendiyse arayüzde de 115200 ile bağlanmak gerekir; hızı bağlandıktan sonra değiştirirsin. 115200 dışındaki ölçümler **S3_b230400.csv** gibi ayrı dosyalara yazılır, ödev ölçümlerine karışmaz.

9. PC arayüzü nasıl çalışıyor

Arayüz iki Python dosyasından oluşuyor. **uart_protocol.py** pencereden bağımsız: mesajları ayrıştırır, CSV yazar, özet istatistik hesaplar. **uart_monitor.py** ise pencere (Tkinter) ve grafikler (matplotlib).

9.1 Neden iki iş parçacığı (thread)?

Pencere sürekli kendini çizmek ve tıklamalara cevap vermek zorunda. Seri porttan veri beklerken pencereyi bekletirsek uygulama donar. Çözüm, kartta kullandığımız kalıbın aynısı:

Kart	PC
Buton ISR (hızlı, kısa)	SerialReader thread'i: portu okur, \n ile satırlara böler
buttonQ	Python queue.Queue
ButtonTask (asıl iş)	Pencere: 50 ms'de bir kuyruğu boşaltır, mesajları işler, ekrana yazar

```
def poll_queue(self):
    try:
        for _ in range(MAX_PER_POLL):
            kind, data = self.rx_q.get_nowait()
            if kind == "line":
                self.handle_frame(data)
    except queue.Empty:
        pass
    except Exception as e:
        self.add_line(f"ARAYÜZ HATASI (okuma döngüsü): {e}", "bad")
    finally:
        self.after(POLL_MS, self.poll_queue) # ne olursa olsun yeniden zamanla
```

Buradaki **finally** önemli bir düzeltme: önceki sürümde bir mesaj işlenirken hata çıkınca yeniden zamanlama satırına ulaşılmıyordu ve arayüz karttan gelen **hiçbir şeyi** okumaz hale geliyordu (Bölüm 11).

9.2 Arayüzün özellikleri

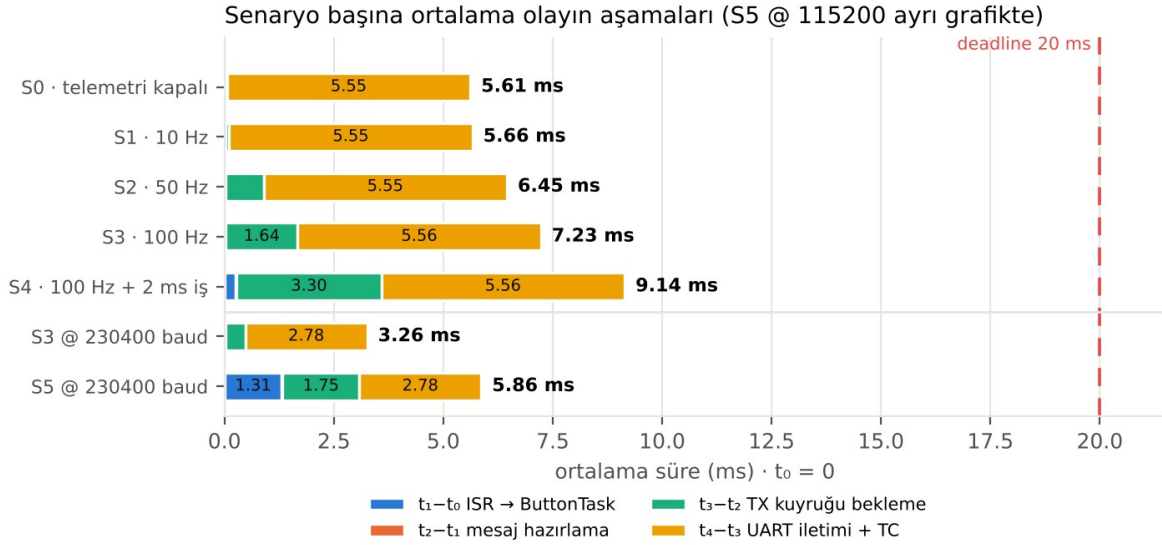
- Port ve baud seçimi, bağlan/kes. Baud değişimi Bölüm 8'deki el sıkışmayla.
- Senaryo butonları (içerikleriyle: "S5 · 100 Hz + 5 ms iş") ve "Deneyi bitir ve kaydet".
- BTN gelince büyük **"Butona basıldı · Olay N"** göstergesi; ısınma sayacı (5 s) ve basış sayacı (x / 30).
- **Olay kayıtları** sekmesi: her basış bir satır; deney bitince t_0 ve dört aşama dolar. 20 ms'yi aşanlar kırmızı, kayıplar kırmızı zeminli.
- **Grafikler**: son deneyin olay → R grafiği (20 ms çizgisiyle) ve ortalama olayın zaman çizelgesi; yakınlaştırma aracı.
- **Özet tablo, Senaryolar** (CPU ve UART yükü hesabı) ve **Zaman aralıkları** ($t_0 \dots t_4$ açıklamaları) sekmeleri.
- Kayıt: **measurements/S3.csv** (şartname biçimi), **summary.csv**, **raw/S3_<tarih>.log**. Eski kopyalardan en yeni 3'ü tutulur.
- D'den sonra 5 s içinde kayıt gelmezse uyarı ve D'yi yeniden gönderme teklifi.

10. Ölçüm sonuçları ve yorum

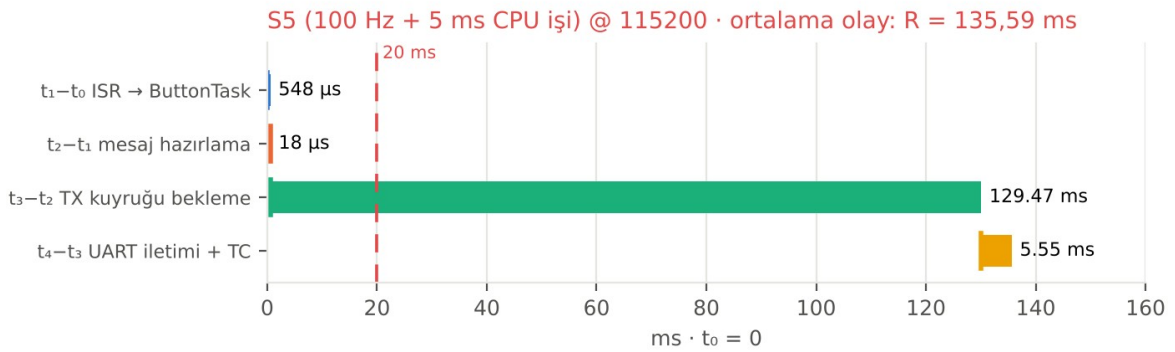
Bu bölümdeki ortalamalar arayüzün özet ekranından alındı (S0–S5 ve 230400 ölçümleri) ve 60 başırlık S3 kaydı doğrudan karttan gelen REC satırlarından hesaplandı. Teslim raporunda tüm sayılar `measurements/*.csv` dosyalarından analiz betiğiyle yeniden üretilecek; S5 ölçümü de düzeltilmiş firmware ile tekrarlanacak.

Senaryo	t_1-t_0	t_2-t_1	t_3-t_2	t_4-t_3	R ort.	Yorum
S0 · kapalı	0,013	0,019	0,028	5,554	5,61	Referans: yalnız hat süresi
S1 · 10 Hz	0,013	0,018	0,077	5,555	5,66	Hat %5,6 dolu, nadiren bekleme
S2 · 50 Hz	0,013	0,018	0,865	5,554	6,45	Hat %27,8 dolu
S3 · 100 Hz	0,012	0,019	1,640	5,557	7,23	Hat %55,6 dolu: TX sırası büyür
S4 · +2 ms iş	0,263	0,018	3,302	5,559	9,14	CPU beklemesi de devrede
S5 · +5 ms iş	0,548	0,018	129,469	5,554	135,59	Aşırı yük: kuyruk birikiyor
S3 @ 230400	0,012	0,018	0,448	2,780	3,26	Hat süresi yarıya indi
S5 @ 230400	1,310	0,018	1,755	2,781	5,86	Tıkanma tamamen kalktı

Tüm süreler ms cinsinden ortalamadır.



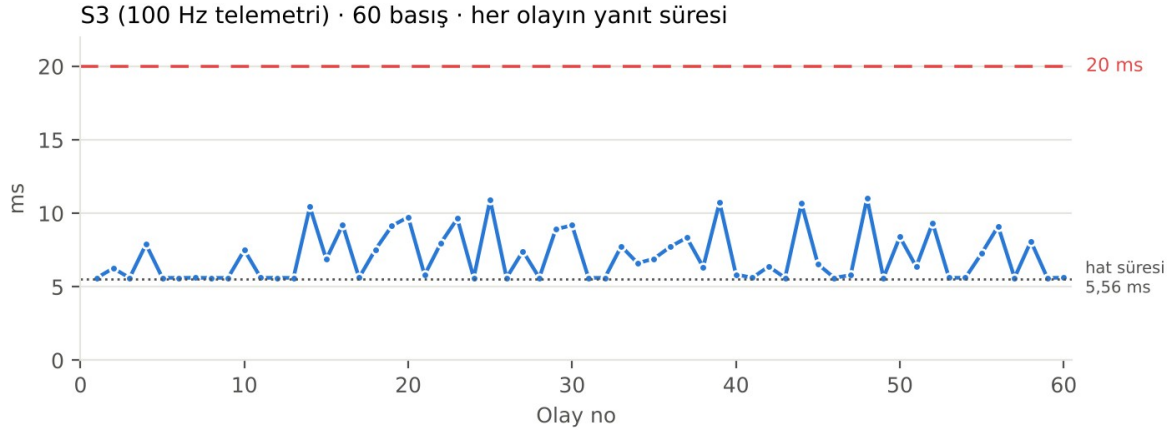
Şekil 2 · Senaryo başına ortalama olayın aşamaları (S5 @ 115200 hariç)



Şekil 3 · S5 @ 115200: TX kuyruğunda bekleme her şeyi bastırıyor

10.1 S0 → S3: Gecikme UART paylaşımından geliyor

t_1-t_0 ve t_2-t_1 tüm bu senaryolarda $\sim 12 \mu s$ ve $\sim 18 \mu s$ ile sabit; ButtonTask öncelikli olduğu için hemen çalışıyor. t_4-t_3 de hat süresine eşit. **Değişen tek aşama t_3-t_2** : butona bastığında UART o an bir TEL gönderiyorsa BTN onun bitmesini bekliyor. Telemetri sıklığı arttıkça hat daha çok meşgul ve bekleme olasılığı artıyor. 60 basışlık S3 deneyinde t_3-t_2 0,027 ms ile 5,435 ms arasında değişti; üst sınır neredeyse tam bir mesajın hat süresi. R en fazla 11,02 ms, **hiçbiri 20 ms'yi aşmadı**.



Şekil 4 · S3, 60 basış: R, 5,6 ms (hat boş) ile ~ 11 ms (bir TEL'in ardında) arasında

10.2 S4: CPU yükü devreye giriyor

TelemetryTask en yüksek öncelikte ve her periyotta 2 ms hesap yapıyor. Butona tam bu sırada basarsan ButtonTask hesap bitene kadar bekliyor: t_1-t_0 ortalaması $12 \mu s$ 'den $263 \mu s$ 'ye çıktı (tek tek olaylarda 0,7 ms'ye kadar). Ayrıca UartTxTask da hesap sırasında CPU alamadığından t_3-t_2 büyüyor. Yine de deadline karşılanıyor.

10.3 S5: Sistem kararsız hale geliyor

Asıl ilginç sonuç burada. UART hattı aslında %55,6 dolu, yani boş vakti var. Ama **UartTxTask en düşük öncelikte**: TelemetryTask'ın 5 ms'lik hesabı sürerken yeni bir gönderim başlatamıyor. Bir mesaj bittiğinde (TC) görev uyanıyor ama hesap bitene kadar Ready durumda bekliyor. Sonuç: her 10 ms'lik periyotta ancak **bir** mesaj gönderilebiliyor. Telemetri de periyotta bir mesaj ürettiği için sistemin hiç payı kalmıyor; her BTN kuyruğa kalıcı bir fazlalık ekliyor ve bu fazlalık hiç erimiyor. Kuyruk 16'ya dolunca mesajlar düşmeye başlıyor.

Kararlılık koşulu

Her periyotta UartTxTask'ın yeni gönderim başlatabildiği boşluk **periyot - iş süresi** kadar. Bu boşluk bir mesajın hat süresinden kısaysa kuyruk erimez:

Kararlılık için: $\text{periyot} - \text{iş süresi} \geq \text{hat süresi}$

S4 @ 115200: $10 - 2 = 8,00 \text{ ms} \geq 5,56 \text{ ms} \rightarrow \text{kararlı}$
S5 @ 115200: $10 - 5 = 5,00 \text{ ms} < 5,56 \text{ ms} \rightarrow \text{KARARSIZ (kuyruk birikir)}$
S5 @ 230400: $10 - 5 = 5,00 \text{ ms} \geq 2,78 \text{ ms} \rightarrow \text{kararlı}$

Bu formül **ölçümle doğrulandı**: baud'u 230400'e çıkarınca hat süresi 2,78 ms'ye indi ve S5'in ortalama yanıtı 135,59 ms'den **5,86 ms**'ye düştü. S5 @ 230400'de t_1-t_0 'ın 1,31 ms olması da beklenen: kuyruk birikmeyince görünür hale gelen tek gecikme, ButtonTask'ın hesap bitmesini beklemesi.

Rapor için ana cümle: **S5'te gecikmenin kaynağı UART hattının kapasitesi değil, en düşük öncelikli UartTxTask'ın CPU'ya erişememesidir**. Etkin TX kapasitesi telemetri üretim hızına eşitleniyor ve her BTN mesajı kalıcı bir birikme yaratıyor.

11. Karşılaştığımız hatalar ve dersleri

Bu bölüm, şartnamenin "kodu nasıl kontrol ettiniz" sorusunun cevabı. Her hatayı **tahmin ederek değil, ölçerek** bulduk; bunu mümkün kılan, her hata yoluna koyduğumuz sayaçlardı.

Belirti	Gerçek neden	Nasıl bulundu / çözüldü	Ders
Arayüzde hiçbir şey yok	USART2 kesmesi hiç üretilmemiş (NVIC → Code generation tikleri)	Sayaçlar: <code>tx_timeout</code> artıyor, <code>tx_err 0</code> → gönderim başlıyor ama kesme gelmiyor	CubeMX ayarı koda yansıdı mı, üretilen dosyada kontrol et
Sıra no sürekli 1'e dönüyor, bozuk satırlar	Siyah RESET butonuna basılıyordu	Reset ↔ çökme farkı: reset'te sayaç sıfırlanır, çökmede akış durur	Belirtiyi doğru oku
10 basışta 25 BTN	Bırakma gürültüsü 30 ms'den sonra da sürüyor	Basılı tutma deneyi + 3 aşamalı filtre (6.3)	Debounce donanıma göre ölçülerek ayarlanır
Ayar değişti ama davranış aynı	CubeMX'te kod yeniden üretilmemiş	<code>main.c</code> 'de <code>GPIO_MODE_IT_RISING_FALLING</code> satırına bakıldı	Değişikliği kaynağında doğrula
S5'te D'ye basınca kayıtlar gelmiyor	Komut TX kuyruğuna konuyordu; kuyruk 16/16 doluyken kayboluyordu	Komut ayrı değişkene (<code>g_pending_cmd</code>) alındı	Kontrol kanalını veri kuyruğuyla paylaşma
Bir süre sonra kart komutları duymuyor	HAL kilit yarışı: gönderim sırasında gelen baytta alım yeniden kurulamıyor	<code>rx_keeplive</code> + <code>rx_yeniden</code> sayacı	Kütüphanenin iç kilitlerini bil
END geldi ama kayıt yok, sonra hiçbir komuta cevap yok	Arayüzün okuma döngüsü bir hata sonrası kendini yeniden zamanlamıyordu	Senin log'undaki eksik "Kaydedildi" satırı; <code>finally</code> + sürüm kontrolü	Döngünün devamını <code>finally</code> 'ye koy; hataları gizleme

Son iki düzeltme (Adım 12 firmware ve okuma döngüsü) donanımda henüz doğrulanmadı. Bir sonraki oturumun ilk işi bu.

12. İyileştirme önerileri

Şartname TX kuyruğunu FIFO ve öncelikleri $3 > 2 > 1$ olarak sabitliyor; ana ölçümler bu yüzden olduğu gibi yapıldı. Aşağıdakiler raporun "tartışma" bölümüne girecek öneriler:

Öneri	Beklenen etki	Bedeli
BTN'i kuyruğun önüne koymak (<code>xQueueSendToFront</code>) + TEL'in kuyruğu tamamen doldurmasını engellemek (ör. 4 yer BTN'e ayrılır)	S5'te en kötü $R \approx 5,5 + 4,5 + 5,6 \approx 15-16 \text{ ms} < 20 \text{ ms}$	Yük altında bazı TEL mesajları düşer; bilinçli bir tercih: kritik olay önce
UartTxTask'ın önceliğini yükseltmek	TC sonrası gönderim gecikmeden başlar, kapasite hatta eşitlenir	Öncelik ilişkisi değişir (şartname dışı)
DMA ile gönderim	Bayt başına kesme yükü kalkar	TC'nin ayrıca beklenmesi gerekir
Baud'u artırmak	230400'de S5 ölçülerek doğrulandı: $135,6 \rightarrow 5,9 \text{ ms}$	Daha hızlı hat; kablo ve dönüştürücü desteklemeli

İlk öneriyi bir derleme anahtarıyla (**TX_BTN_PRIORITY**) ekleyip yalnız S5'i yeniden ölçmek, raporda "önerdim ve ölçtüm" diyebilmeyi sağlar.

13. Öğrenmen gerekenler ve kontrol soruları

Proje boyunca "bunu bilmen önemli" dediğim konuların derli toplu listesi. Her birini kendi cümlelerinle açıklayabiliyorsan videoda ve ders sırasında rahat edersin.

Konu	Bir cümlelik özet
Preemptive scheduling	Hazır olan en yüksek öncelikli görev çalışır; daha öncelikli biri hazır olunca çalışan anında kesilir
Görev durumları	Running, Ready, Blocked, Suspended. Blocked görev CPU harcamaz
Queue	Veriyi kopyalayarak taşıyan FIFO; dolu/boş durumda bloklama ya da vazgeçme
Semaphore vs mutex	Semafor sinyal verir, sahibi yok; mutex kaynağı korur, sahibi ve öncelik kalıtımı var
Task notification	Göreve doğrudan, hafif sinyal; tek alıcı
FromISR + portYIELD_FROM_ISR	Kesmede bloklamayan API; uyanan görev öncelikliyse hemen geçiş
NVIC öncelikleri	Küçük sayı = yüksek öncelik; FreeRTOS çağıran kesmeler ≥ 5 ; SysTick/PendSV = 15
vTaskDelayUntil	Periyodu kaydırmadan sabit tutar
Kritik bölge	Paylaşılan veriyi kısa süre kesmelere karşı korur; çok kısa tutulur
volatile	Kesme ve görev arasında paylaşılan değişkenler; derleyici her seferinde bellekten okur
TXE vs TC	TXE: DR boş; TC: son bit hattın çıktı. Yanıtın bittiği an TC
IT gönderimde tampon ömrü	Gönderim bitene kadar tampon yerinde kalmalı (static)
Stack taşması	Sessizce belleği bozabilir; CHECK_FOR_STACK_OVERFLOW ile yakala
newlib reentrancy	snprintf gibi fonksiyonlar birden çok görevden çağrılacaksa USE_NEWLIB_REENTRANT
uint32 fark aritmetiği	Sayaç başa dönse de (uint32_t)(b - a) doğru süreyi verir
Debounce	Mekanik buton titrer; filtre ölçülerek tasarlanır, atılan kenarlar sayılır
Duvar saati vs CPU süresi	Ölçtüğümüz aralıklar araya giren kesme ve görevleri de içerir

Kontrol soruları

1. Bir REC satırında t_3 ve t_4 neden boş olabilir? (**tx_drop** ne demek?)
2. S0'da telemetri kapalıyken TelemetryTask ne yapıyor, neden boş döngüde dönmüyor?
3. t_1-t_0 neden 1 ms değil de $\sim 12 \mu s$? Hangi satır bunu sağlıyor?
4. UART'ı iki görev doğrudan kullansaydı neye ihtiyacımız olurdu, neden kullanmadık?
5. Mutex'i kesme içinde neden kullanamayız?
6. S3'te t_3-t_2 neden 0 ile $\sim 5,5$ ms arasında rastgele değişiyor? Üst sınır neden $\sim 5,5$ ms?
7. S5'te UART hattı %55 doluyken kuyruk neden birikiyor?
8. Baud değişiminde kart neden önce eski hızla SWITCH gönderip TC'yi bekliyor?

9. `tx_cur` neden yerel değişken değil de `static`?
10. D komutunu TX kuyruğuna koymak neden S5'te sorun çıkardı?

14. Açık kalan işler ve GitHub

İş	Durum
Firmware (Adım 12) ve arayüzün son sürümünü donanımda doğrula: D sonrası kayıt, ardından yeni senaryo komutu	Sıradaki ilk iş
S5'i düzeltilmiş firmware ile ≥ 30 basışla yeniden ölç (<code>tx_drop</code> , <code>tel_drop</code> , <code>txq_max</code> değerleri)	Yapılacak
Tüm ölçümlerde basışlar arası $\geq 0,5$ s (S3-60 deneyinde ortalama $\sim 0,39$ s idi)	Dikkat
Analiz betiği: CSV'lerden özet tablo ve iki grafik (şartname: grafik kodu da depoda)	Yapılacak
<code>analysis/report.md</code> , <code>README.md</code> , <code>docs/setup.md</code> , <code>docs/code-notes.md</code> , <code>docs/ai-usage.md</code>	Bu rapor iyi bir temel
(İsteğe bağlı) BTN öncelikli kuyruk varyantı ile S5 ölçümü	Öneri
GitHub deposu ve kısa anlatım videosu	Yapılacak

Şartnamedeki depo düzeni

```
freertos-bootcamp/
├── hafta-01/
│   ├── README.md
│   ├── firmware/           ← CubeIDE projesi (Core/, .ioc, FreeRTOSConfig.h dahil)
│   ├── interface/         ← uart_monitor.py, uart_protocol.py
│   ├── measurements/      ← S0.csv ... S5.csv, summary.csv, raw/
│   ├── analysis/          ← report.md, plots/, analiz betiği
│   └── docs/               ← protocol.md, setup.md, code-notes.md, ai-usage.md
```

Bir sonraki adımda bu yapıyı birlikte kurup Git ile GitHub'a yükleyeceğiz: depo oluşturma, `.gitignore` (Debug/ derleme çıktıları hariç), ilk commit ve teslim için commit SHA'sı.